Guion de trabajo tutorizado

Ingesta de datos en cloud

Plataformas Tecnológicas

Profesor D. Alejandro Gómez Morón
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «Ingesta de datos en cloud - TrabajoTutorizado.pdf» (Campus Virtual UPO).

Objetivos

- Afianzar los conocimientos adquiridos en clase.
- Aplicar la tecnología de ingesta de datos para resolver problemas.
- Diseñar una arquitectura que abarque necesidades de ingesta de datos en la nube.
- Conocer algunas de las tecnologías usadas en ingesta de datos.

Actividades

Las preguntas deberá contestarlas razonadas con links a cualquier documentación usada y útil y, para los problemas, vendrá en cada uno lo que se requiere.

Preguntas

Pregunta 1 (30 minutos 1 punto) En clase se ha visto el concepto de factor de replicación en Kafka. Se estableció como ideal el número 3. ¿Pero es siempre el ideal? ¿En qué casos se le ocurre que podríamos querer aumentar o disminuir dicho factor? ¿Cuáles son las ventajas y las desventajas?

Pregunta 2 (30 minutos 1 punto) ¿Cómo se puede cambiar ese número de replicación de Kafka?

Pregunta 3 (30 minutos 2 puntos) Supongamos que, al igual que hemos visto en clase el sistema que genera thumbnails de una fotografía, queremos también guardar los metadatos de ésta. Supongamos que se nos indica que hay un servicio que, desde la Cloud Function o Lambda que implementemos, si mandamos la fotografía, nos devuelve sus metadatos y estos tenemos que guardarlos en otro Google Storage / S3 en formato .json con el nombre de la fotografia con el patrón “nombreFoto_metadata.json” junto con “nombreFoto_thumnail”. Diseñe un diagrama con qué componentes usaría (puede usar los productos de Google o AWS, lo que prefiera para diseñarlo, es decir, S3 + Lambdas + lo que necesite o Google Storage + Cloud Function + lo que necesite).

Ejercicios

Ejercicio 1 (75 minutos - 3 puntos).- En clase vimos un ejemplo de Kafka con productor y consumidor. Se pide que, ese ejemplo, lo implemente y ejecute tanto el productor como el consumidor y describa el flujo que ha ocurrido (primero ejecute el producer.py y luego el consumer.py).

1) ¿Qué salida obtiene?

2) Esto que ha implementado, ¿es batching o streaming?

Ejercicio 2 (75 minutos - 3 puntos).- Modifique el ejemplo anterior pero, en este caso, queremos tener 2 producers, pero uno que envíe frases de una película/serie/saga que os guste como en el ejemplo y, otro, que otra película que le guste y a otro topic diferente - busque cómo hacerlo -, pero ambos enviando simultáneamente frases (tendremos dos scripts ejecutándose de producers). También elimine los 3s de pausa que hay y modifíquelo por 2s, haga capturas. En cambio, vamos a dejar 1 consumer.

1) ¿Qué salida obtiene?

2) ¿Qué frases salen en el consumer? ¿Por qué?

Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- La entrega se ha de llevar a cabo durante el día 2 de febrero de 2026.

Bibliografía

1. Apache Kafka: https://kafka.apache.org/

2. Apache Beam: https://beam.apache.org/

3. Pub/Sub: https://cloud.google.com/pubsub

RESOLUCIÓN

Pregunta 1. ¿Es 3 siempre el factor de replicación ideal en Kafka?

El factor de replicación (RF) indica cuántas copias de cada partición mantiene el clúster: una actúa como *líder* y las demás como *followers*. RF = 3 es el valor recomendado por defecto porque, combinado con `min.insync.replicas=2` y `acks=all`, permite perder un broker sin detener la escritura y sin perder mensajes confirmados. Aun así **no es un óptimo universal**: es un compromiso entre durabilidad, disponibilidad, coste y latencia.

Situación	RF recomendado	Motivo
Desarrollo, pruebas, clúster de 1 nodo (como el de este trabajo)	1	No hay más brokers; replicar no aporta nada y consume disco.
Datos regenerables o de bajo valor (logs de depuración, métricas efímeras, topics intermedios que se pueden recalcular)	1-2	Se prioriza coste y rendimiento; perder datos es tolerable.
Producción estándar	3	Tolera la caída de 1 broker (o 1 zona de disponibilidad) manteniendo escrituras con min.insync.replicas=2.
Datos críticos (pagos, eventos regulatorios, auditoría) o despliegues multi-AZ / multi-rack	4-5	Tolera 2 fallos simultáneos o un mantenimiento más un fallo; útil con 3 AZ más réplicas extra.
Servicios gestionados (Confluent Cloud, MSK, Event Hubs)	Fijado por el proveedor (normalmente 3)	La durabilidad la garantiza el SLA del servicio.

Ventajas de aumentarlo: mayor durabilidad y disponibilidad, más tolerancia a fallos de nodo o zona, y mantenimientos (rolling restarts) sin pérdida de servicio.

Desventajas de aumentarlo: el almacenamiento se multiplica por RF; hay más tráfico de red entre brokers; con `acks=all` la latencia de escritura sube, porque el líder espera a más réplicas; y RF no puede superar el número de brokers. **Disminuirlo** abarata y acelera, pero con RF = 1 la caída del broker que aloja una partición deja esos datos inaccesibles o perdidos.

Pregunta 2. ¿Cómo se cambia el factor de replicación?

- **Para topics nuevos:** se indica al crearlos (`kafka-topics.sh --create --topic t --partitions 3 --replication-factor 3`) o se fija un valor por defecto en el broker con `default.replication.factor=3` en `server.properties`. Los topics internos tienen sus propios parámetros (`offsets.topic.replication.factor`, `transaction.state.log.replication.factor`), que en el `docker-compose.yml` de clase están a 1.
- **Para topics ya existentes:** `kafka-topics.sh --alter` **no** permite cambiar el RF (solo aumenta particiones). Hay que hacer una *reasignación de particiones*: se escribe un JSON con la nueva lista de réplicas de cada partición y se ejecuta `kafka-reassign-partitions.sh --bootstrap-server kafka:9092 --reassignment-json-file incrementar.json --execute`, y después `--verify` para comprobarlo. Herramientas como Cruise Control o la API AdminClient (`alterPartitionReassignments`) automatizan este paso.

- Conviene revisar también `min.insync.replicas` (a nivel de broker o de topic) para que sea coherente con el nuevo RF, típicamente RF − 1.

```
{"version":1,
 "partitions":[
   {"topic":"wordcount","partition":0,"replicas":[1,2,3]}
 ]}
```

El estado real del topic `wordcount` en el broker KRaft de esta práctica (1 nodo, RF = 1):

```
aramber@localhost:~$ kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --create --topic wordcount --partitions 1 --replication-factor 1
Created topic wordcount.
aramber@localhost:~$ kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --describe --topic wordcount
Topic: wordcount    TopicId: qtDESH2XSe-GbWogQStNDw    PartitionCount: 1    ReplicationFactor: 1    Configs: segment.bytes=1073741824
    Topic: wordcount    Partition: 0    Leader: 1    Replicas: 1    Isr: 1    Elr:    LastKnownElr:
```

Figura 1. Creación y descripción del topic `wordcount` en el broker Kafka 3.9.1 (modo KRaft). Replicas: 1, Isr: 1.

Pregunta 3. Diseño: miniatura + metadatos de cada fotografía

Propuesta sobre Google Cloud, equivalente a S3 + Lambda + SNS/SQS en AWS. La subida de `foto.jpg` al bucket de entrada genera un evento *object finalized*. Eventarc lo publica en un topic de Pub/Sub, que funciona como *fan-out* hacia dos Cloud Run Functions independientes:

- `thumbnail()` genera la miniatura con Pillow y la guarda como `nombreFoto_thumbnail.jpg`.
- `metadata()` envía la foto al servicio externo de metadatos, recibe el JSON y lo guarda como `nombreFoto_metadata.json`.
- Ambas escriben en un **bucket distinto** del de entrada. Así se evita que la escritura vuelva a disparar el trigger (bucle infinito).
- Separar las dos funciones hace que un fallo o una latencia alta del servicio externo no bloquee las miniaturas. Además, cada una escala y se reintenta por separado; tras N fallos, el mensaje va a una *dead-letter queue*.
- Cloud Logging y Monitoring recogen trazas y alertas. Las funciones usan una cuenta de servicio con permisos mínimos: lectura en el bucket de entrada y escritura en el de salida.

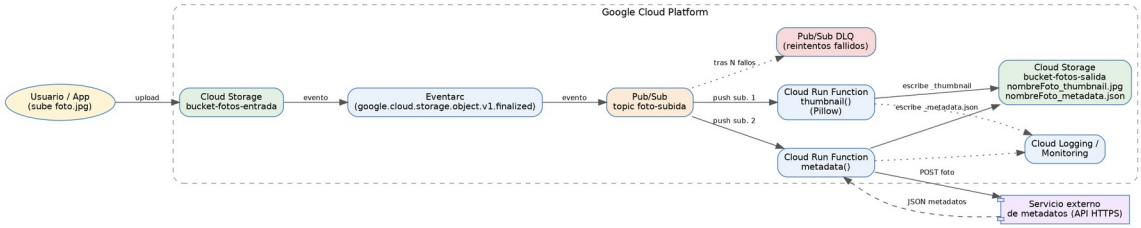


Figura 2. Arquitectura propuesta (Cloud Storage → Eventarc → Pub/Sub → 2 Cloud Run Functions → Cloud Storage de salida).

Equivalente en AWS: S3 (evento `ObjectCreated`) → SNS → dos colas SQS → Lambda `thumbnail` y Lambda `metadata` → bucket S3 de salida, con DLQ en SQS y logs en CloudWatch.

Ejercicio 1. Productor y consumidor Kafka del ejemplo de clase

Se ha usado el código de `kafka.zip` (carpeta `KRaft`) sin modificar. El broker es Apache Kafka 3.9.1 en modo KRaft (sin ZooKeeper), el mismo modo que la imagen `confluentinc/cp-kafka:7.5.0` del `docker-compose.yml`, escuchando en `kafka:9092`. Primero se lanza `producer.py`; después, `consumer.py`.

```
aramber@localhost:~$ python3 producer.py      # terminal 1 (Ctrl+C a los 20 s)
Conectado a Kafka
Producing: Happiness can be found even in the darkest of times
Producing: It is our choices that show what we truly are
Producing: We are only as strong as we are united
Producing: We are only as strong as we are united
Producing: Happiness can be found even in the darkest of times
Producing: Fear of a name increases fear of the thing itself
Producing: Fear of a name increases fear of the thing itself
^C
```

Figura 3. Ejecución de `producer.py`: se conecta a Kafka y publica una frase aleatoria cada 3 s en el topic `wordcount`.

```
aramber@localhost:~$ python3 consumer.py      # terminal 2
Conectado a Kafka
Esperando mensajes...
Recibido: Happiness can be found even in the darkest of times
Recibido: It is our choices that show what we truly are
Recibido: We are only as strong as we are united
Recibido: We are only as strong as we are united
Recibido: Happiness can be found even in the darkest of times
Recibido: Fear of a name increases fear of the thing itself
Recibido: Fear of a name increases fear of the thing itself
[... salida de conteo intermedio omitida ...]
Conteo actual: {
  "happiness": 2,
  "can": 2,
  "be": 2,
  "found": 2,
  "even": 2,
  "in": 2,
  "the": 4,
  "darkest": 2,
  "of": 6,
  "times": 2,
  "it": 1,
  "is": 1,
  "our": 1,
  "choices": 1,
  "that": 1,
  "show": 1,
  "what": 1,
  "we": 5,
  "truly": 1,
  "are": 5,
  "only": 2,
  "as": 4,
  "strong": 2,
  "united": 2,
  "fear": 4,
  "a": 2,
  "name": 2,
  "increases": 2,
  "thing": 2,
  "itself": 2
}

^C
```

Figura 4. Ejecución posterior de `consumer.py`: lee desde el offset más antiguo (`auto_offset_reset=earliest`) las 7 frases que ya estaban en el topic y mantiene un conteo acumulado de palabras.

1) ¿Qué salida se obtiene? Descripción del flujo

- El productor abre una conexión con el broker (`bootstrap_servers=kafka:9092`) y, en un bucle infinito, elige una de 4 frases, la codifica en UTF-8 y hace `send()` al topic `wordcount`. El broker la añade al final del log de la partición 0 con un offset creciente (0, 1, 2…).
- Como el consumidor se ejecuta **después**, las frases quedan persistidas en el log mientras tanto. Al arrancar con el `group_id` `wordcount-group`, que no tiene offsets guardados, `auto_offset_reset='earliest'` le hace empezar en el offset 0. Por eso recibe las 7 frases producidas antes, en el mismo orden.
- Por cada mensaje, el consumidor imprime `Recibido: …`, divide la frase en palabras en minúsculas y actualiza un `Counter`, que muestra como JSON. El conteo final coincide con las frases enviadas; por ejemplo, «we» aparece 5 veces (2 × «We are only as strong as we are united» + 1 × «…what we truly are»).
- El consumidor confirma periódicamente sus offsets (*auto-commit*). Si se relanza, continúa donde lo dejó en lugar de repetir mensajes.

2) ¿Batching o streaming?

Es **streaming** (procesamiento de eventos en tiempo real): cada frase se publica en cuanto se genera y el consumidor la procesa **evento a evento**, manteniendo un estado incremental (el conteo) sobre un flujo no acotado. Que en este experimento el consumidor arranque más tarde y encuentre mensajes acumulados no lo convierte en batch: Kafka actúa como *buffer* persistente y, si ambos se ejecutan a la vez, la latencia entre producción y consumo es de milisegundos, como se ve en el ejercicio 2. Internamente, el cliente agrupa mensajes en *micro-lotes* de red (`linger.ms`, `batch.size`), pero es una optimización de transporte, no un procesamiento por lotes.

Ejercicio 2. Dos productores, dos topics y un consumidor

Cambios sobre el ejemplo:

- `producer_lotr.py` envía frases de *El Señor de los Anillos* al topic `lotr`.
- `producer_starwars.py` envía frases de *Star Wars* al topic `starwars`.
- En ambos productores la pausa pasa de 3 s a **2 s**.
- Un único `consumer.py` se suscribe a los dos topics a la vez: `KafkaConsumer('lotr', 'starwars', …)`. También podría suscribirse con un patrón (`consumer.subscribe(pattern='^(lotr|starwars)$')`).
- Los tres scripts se ejecutan simultáneamente en tres terminales.

```
# producer_starwars.py (fragmento)
frases = ["May the Force be with you", "Do or do not there is no try",
          "I am your father", "Fear is the path to the dark side"]
while True:
    f = random.choice(frases)
    print(f"[topic=starwars] Producing: {f}", flush=True)
    producer.send('starwars', f.encode('utf-8'))
    time.sleep(2)   # antes 3 s
```

```
# consumer.py (fragmento)
c = KafkaConsumer('lotr', 'starwars', bootstrap_servers='kafka:9092',
                  auto_offset_reset='earliest', group_id='wordcount-group-2')
```

```
aramber@localhost:~$ kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --create --topic lotr
Created topic lotr.
aramber@localhost:~$ kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --create --topic starwars
Created topic starwars.
aramber@localhost:~$ kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --list
__consumer_offsets
lotr
starwars
wordcount
```

Figura 5. Creación de los topics `lotr` y `starwars`.

```
aramber@localhost:~$ python3 producer_lotr.py        # terminal 1
Conectado a Kafka
[topic=lotr] Producing: All we have to decide is what to do with the time that is given us
[topic=lotr] Producing: Even the smallest person can change the course of the future
[topic=lotr] Producing: All we have to decide is what to do with the time that is given us
[topic=lotr] Producing: One ring to rule them all
[topic=lotr] Producing: One ring to rule them all
[topic=lotr] Producing: Not all those who wander are lost
[topic=lotr] Producing: One ring to rule them all
[topic=lotr] Producing: All we have to decide is what to do with the time that is given us
^C
```

Figura 6. Terminal 1: productor de El Señor de los Anillos (topic `lotr`), una frase cada 2 s.

```
aramber@localhost:~$ python3 producer_starwars.py    # terminal 2
Conectado a Kafka
[topic=starwars] Producing: Do or do not there is no try
[topic=starwars] Producing: Fear is the path to the dark side
[topic=starwars] Producing: Fear is the path to the dark side
[topic=starwars] Producing: Do or do not there is no try
[topic=starwars] Producing: Fear is the path to the dark side
[topic=starwars] Producing: Fear is the path to the dark side
[topic=starwars] Producing: Fear is the path to the dark side
[topic=starwars] Producing: I am your father
^C
```

Figura 7. Terminal 2: productor de Star Wars (topic `starwars`), ejecutándose simultáneamente.

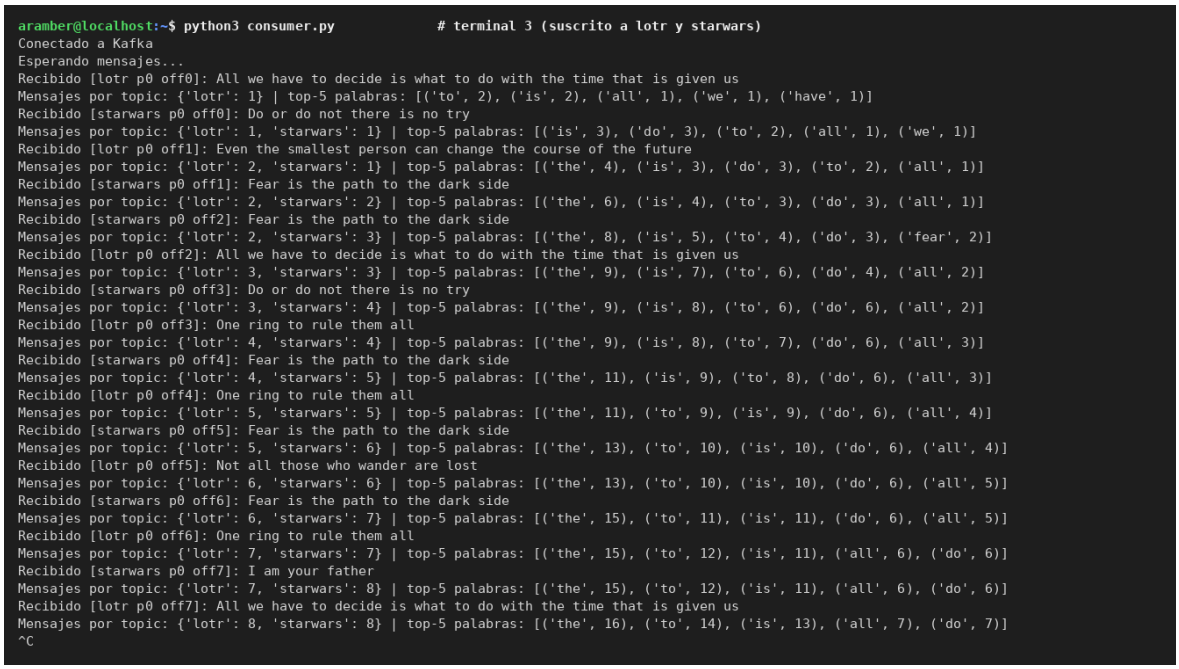


Figura 8. Terminal 3: un único consumidor recibe intercalados los mensajes de ambos topics, indicando topic, partición y offset.

1) ¿Qué salida se obtiene?

Cada productor imprime sus 8 frases, a razón de una cada 2 s durante ~16 s. El consumidor, que ya estaba escuchando, recibe **16 mensajes**, 8 de cada topic, casi en el mismo instante en que se producen. Los muestra **intercalados** (lotr, starwars, lotr, starwars…) con su topic, partición y offset, y actualiza un contador por topic (`{'lotr': 8, 'starwars': 8}`) y el top-5 de palabras de ambas películas combinadas.

2) ¿Qué frases salen en el consumer? ¿Por qué?

Salen **las frases de las dos películas**, porque el consumidor está suscrito a ambos topics. Dentro de cada topic llegan **en el mismo orden** en que se produjeron (offsets 0, 1, 2… de la partición 0): Kafka garantiza el orden dentro de una partición. **Entre topics distintos no hay orden global**; el intercalado depende de cuándo llega cada mensaje y del orden en que el consumidor hace *fetch* de cada partición. Con un solo consumidor en el grupo `wordcount-group-2`, se le asignan todas las particiones de ambos topics. Si hubiera dos consumidores en el mismo grupo, Kafka repartiría las particiones y cada uno vería solo las frases de su partición o topic. Si estuvieran en grupos distintos, ambos recibirían todas las frases (modelo publicación/suscripción).

Referencias

Apache Kafka Documentation – Replication, Broker configs (`default.replication.factor`, `min.insync.replicas`): https://kafka.apache.org/documentation/#replication

Apache Kafka – Increasing replication factor (`kafka-reassign-partitions.sh`): https://kafka.apache.org/documentation/#basic_ops_increase_replication_factor

Google Cloud – Cloud Run functions triggers de Cloud Storage: https://cloud.google.com/functions/docs/calling/storage

Google Cloud – Pub/Sub dead-letter topics: https://cloud.google.com/pubsub/docs/handling-failures

kafka-python-ng – KafkaConsumer: https://github.com/wbarnha/kafka-python-ng